Name: M. Bhavani | **Roll No:** 26000041
Branch & Sec: B.Tech CSE – Section 1 | **Academic Year:** 2026–2030

CASE STUDY 01: Employee Salary & Progressive Deduction Calculator

Question #01

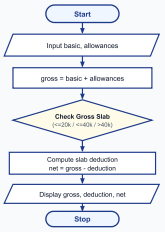
1. PROBLEM STATEMENT: Read an employee's basic salary and allowances, compute gross salary, apply progressive slab deductions, and display net payable salary.

2. OBJECTIVES: Read basic pay and allowances • Compute gross salary • Calculate progressive slab deduction (0%, 5%, 10%) • Output gross, deduction, and net salary.

3. ASSUMPTIONS: Up to Rs. 20,000 has 0 deduction; Rs. 20,001–40,000 is charged 5% above 20,000; above Rs. 40,000 is charged Rs. 1,000 + 10% above 40,000.

4. ALGORITHM: 1. Start → 2. Input basic salary & allowances → 3. Compute gross = basic + allowances → 4. Check gross slab & compute deduction → 5. Compute net = gross − deduction → 6. Display gross, deduction, net → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

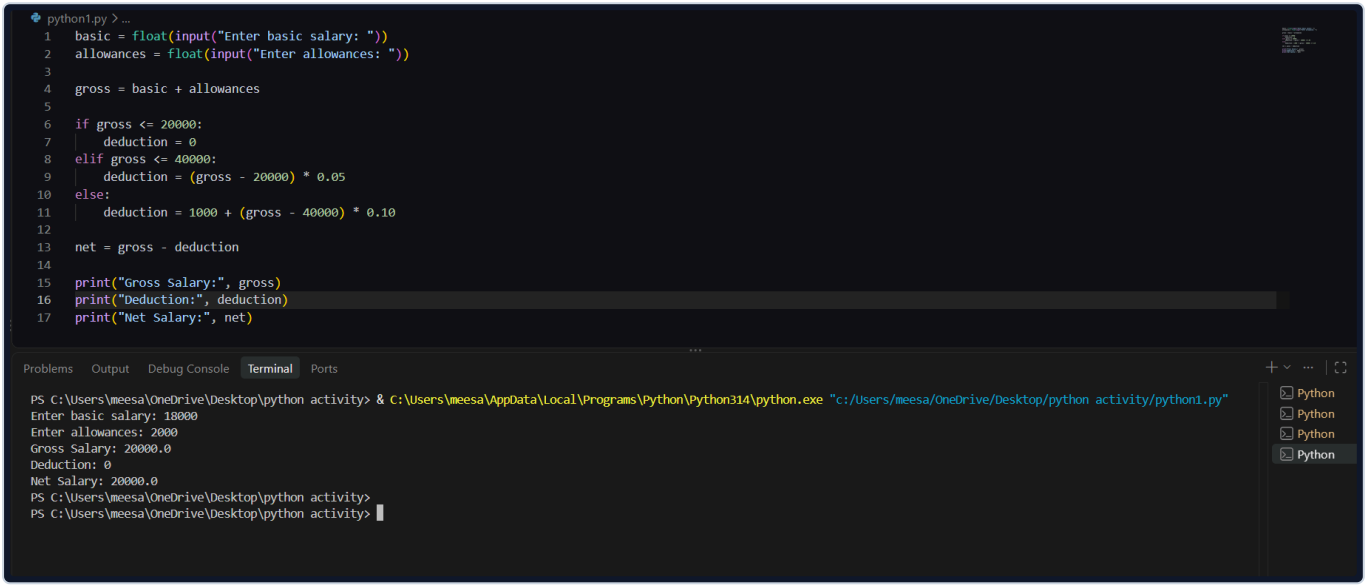
```
BEGIN INPUT basic, allowances gross = basic + allowances IF gross <= 20000 THEN deduction = 0 ELSE IF gross <= 40000 THEN deduction = (gross - 20000) * 0.05 ELSE deduction = 1000 + (gross - 40000) * 0.10 ENDIF net = gross - deduction PRINT gross, deduction, net END
```

7. PYTHON SOURCE CODE

```
basic = float(input("Enter basic salary: ")) allowances = float(input("Enter allowances: ")) gross = basic + allowances if gross <= 20000: deduction = 0 elif gross <= 40000: deduction = (gross - 20000) * 0.05 else: deduction = 1000 + (gross - 40000) * 0.10 net = gross - deduction print("Gross Salary:", gross) print("Deduction:", deduction) print("Net Salary:", net)
```

8. TEST CASE: **Sample Input:** Basic Salary = 18000, Allowances = 2000 | **Expected Output:** Gross Salary = 20000.0, Deduction = 0, Net Salary = 20000.0.

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program accurately calculates gross pay, applies multi-slab deductions using conditional

CASE STUDY 02: Student Attendance & 75% Eligibility Calculator

Question #02

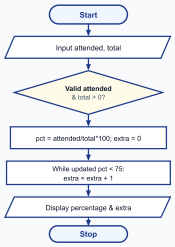
1. PROBLEM STATEMENT: Validate attended and total classes, compute current attendance percentage, and find the minimum consecutive extra classes required to reach 75%.

2. OBJECTIVES: Read classes attended and total classes • Validate non-zero and logical bounds • Calculate current percentage • Use a while loop to count extra classes needed for 75%.

3. ASSUMPTIONS: Total classes must be > 0 and 0 ≤ attended ≤ total. Each extra class attended increases both attended count and total class count by 1.

4. ALGORITHM: 1. Start → 2. Input attended & total → 3. If total==0 or attended<0 or attended>total, print "Invalid input" → 4. Else compute percentage = attended/total*100 → 5. Increment extra while (attended+extra)/(total+extra)*100 < 75 → 6. Print percentage & extra → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

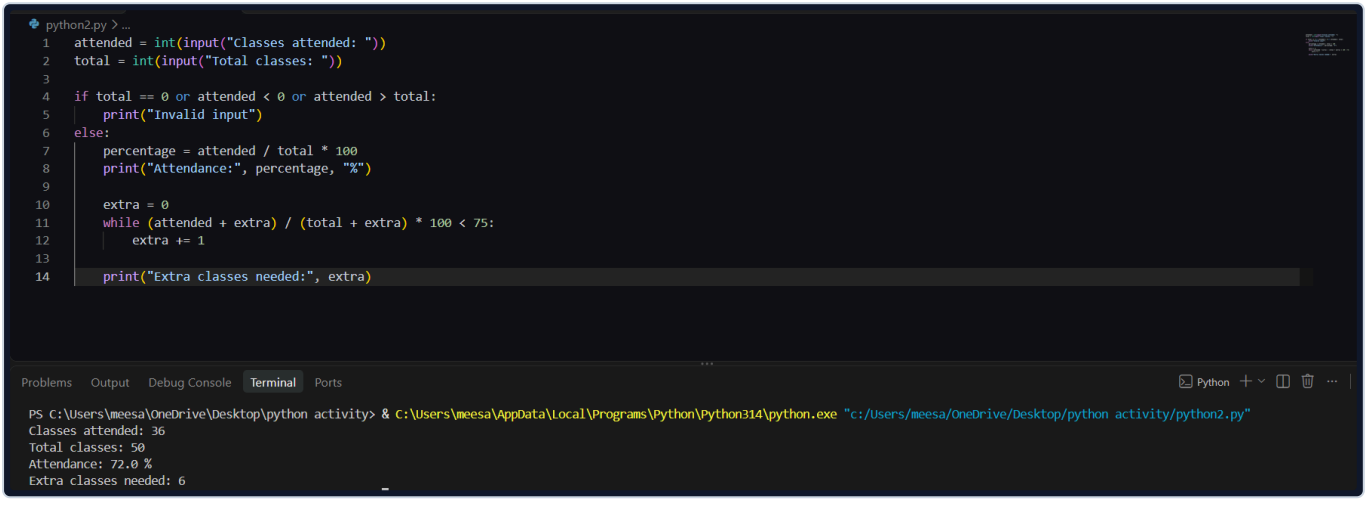
```
BEGIN INPUT attended,
total IF total == 0 OR attended <
0 OR attended > total THEN PRINT
"Invalid input" ELSE pct =
(attended / total) * 100 PRINT pct
extra = 0 WHILE
((attended+extra)/(total+extra))*1
00 < 75 DO extra = extra + 1
ENDWHILE PRINT extra ENDIF END
```

7. PYTHON SOURCE CODE

```
attended = int(input("Classes attended: ")) total
= int(input("Total classes: ")) if total == 0 or attended
< 0 or attended > total: print("Invalid input") else:
percentage = attended / total * 100 print("Attendance:",
percentage, "%") extra = 0 while (attended + extra) /
(total + extra) * 100 < 75: extra += 1 print("Extra
classes needed:", extra)
```

8. TEST CASE: **Sample Input:** Classes attended = 36, Total classes = 50 | **Expected Output:** Attendance: 72.0 %, Extra classes needed: 6.

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program validates class counts, computes attendance percentage, and iteratively determines the exact extra classes required to attain 75%.

CASE STUDY 03: Bank Account Deposit & Withdrawal Validator

Question #03

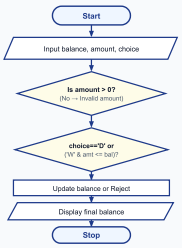
1. PROBLEM STATEMENT: Process a single bank account transaction (deposit or withdrawal) after validating positive transaction amount and sufficient balance.

2. OBJECTIVES: Read opening balance, transaction amount, and operation code ('D' or 'W') • Reject non-positive amounts • Check overdraft on withdrawal • Display final balance.

3. ASSUMPTIONS: 'D' denotes Deposit and 'W' denotes Withdrawal (case-sensitive as per code). Overdraft is not permitted (withdrawal amount must be ≤ balance).

4. ALGORITHM: 1. Start → 2. Input balance, amount, choice → 3. If amount ≤ 0 print "Invalid amount" → 4. Else if choice=="D" add amount to balance → 5. Else if choice=="W" and amount ≤ balance subtract amount → 6. Else print "Transaction rejected" → 7. Print final balance → 8. Stop.

5. FLOWCHART



6. PSEUDOCODE

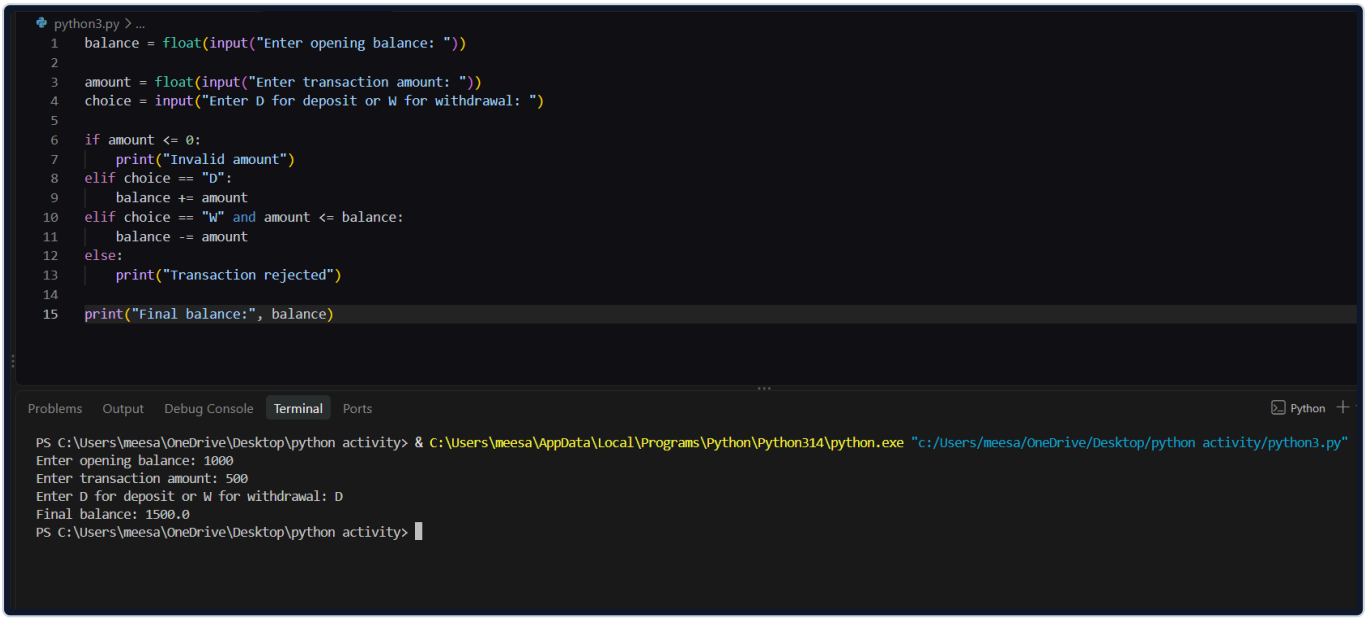
```
BEGIN INPUT balance,
amount, choice IF amount <= 0 THEN
PRINT "Invalid amount" ELSE IF
choice == "D" THEN balance =
balance + amount ELSE IF choice ==
"W" AND amount <= balance THEN
balance = balance - amount ELSE
PRINT "Transaction rejected" ENDIF
PRINT "Final balance:", balance
END
```

7. PYTHON SOURCE CODE

```
balance = float(input("Enter opening balance: "))
amount = float(input("Enter transaction amount: "))
choice = input("Enter D for deposit or W for withdrawal:
") if amount <= 0: print("Invalid amount") elif choice ==
"D": balance += amount elif choice == "W" and amount <=
balance: balance -= amount else: print("Transaction
rejected") print("Final balance:", balance)
```

8. TEST CASE: **Sample Input:** Opening balance = 1000, Transaction amount = 500, Choice = D | **Expected Output:** Final balance: 1500.0.

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program validates positive amounts and sufficient funds before processing deposits or withdrawals and prints the updated balance.

CASE STUDY 04: Inventory Stock Reconciliation & Replenishment

Question #04

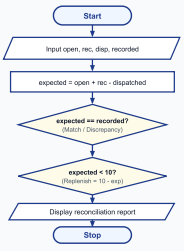
1. PROBLEM STATEMENT: Compute expected closing stock from opening, received, and dispatched stock, reconcile against recorded stock, and flag low-stock replenishment.

2. OBJECTIVES: Read opening, received, dispatched, and recorded quantities • Calculate expected stock • Detect stock discrepancy • Alert replenishment if expected < 10.

3. ASSUMPTIONS: Expected closing stock = opening + received − dispatched. Minimum threshold is 10 units; if below 10, replenishment needed = 10 − expected.

4. ALGORITHM: 1. Start → 2. Input opening, received, dispatched, recorded → 3. expected = opening + received − dispatched → 4. Print expected → 5. Compare expected == recorded → 6. If expected < 10 print replenishment needed → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

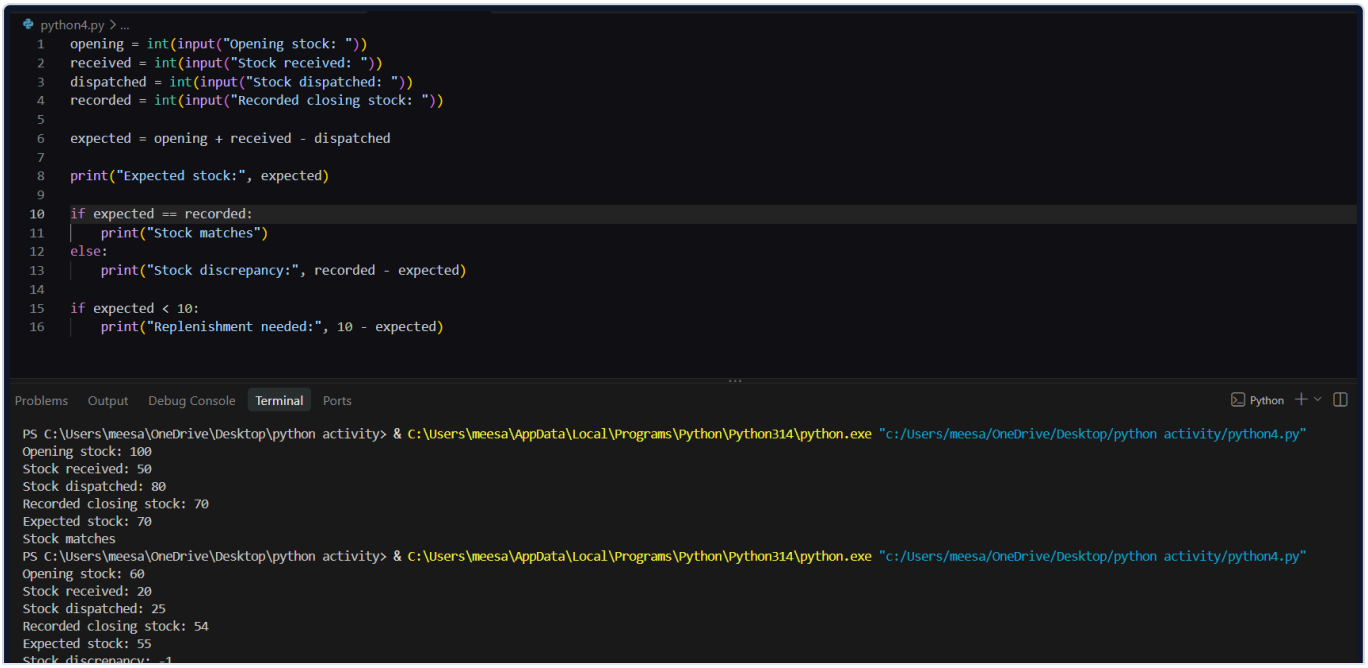
```
BEGIN INPUT opening, received, dispatched, recorded
expected = opening + received - dispatched PRINT "Expected stock:", expected IF expected == recorded THEN PRINT "Stock matches" ELSE PRINT "Stock discrepancy:", recorded - expected ENDIF IF expected < 10 THEN PRINT "Replenishment needed:", 10 - expected ENDIF END
```

7. PYTHON SOURCE CODE

```
opening = int(input("Opening stock: ")) received = int(input("Stock received: ")) dispatched = int(input("Stock dispatched: ")) recorded = int(input("Recorded closing stock: ")) expected = opening + received - dispatched print("Expected stock:", expected) if expected == recorded: print("Stock matches") else: print("Stock discrepancy:", recorded - expected) if expected < 10: print("Replenishment needed:", 10 - expected)
```

8. TEST CASE: **Sample Input:** Run 1: Open=100, Rec=50, Disp=80, Rec=70 (`Stock matches`) | Run 2: Open=60, Rec=20, Disp=25, Rec=54 (Expected: 55, Discrepancy: -1).

9. CODE AND OUTPUT SCREENSHOT



```
python4.py > ...
1  opening = int(input("Opening stock: "))
2  received = int(input("Stock received: "))
3  dispatched = int(input("Stock dispatched: "))
4  recorded = int(input("Recorded closing stock: "))
5
6  expected = opening + received - dispatched
7
8  print("Expected stock:", expected)
9
10 if expected == recorded:
11     print("Stock matches")
12 else:
13     print("Stock discrepancy:", recorded - expected)
14
15 if expected < 10:
16     print("Replenishment needed:", 10 - expected)

Problems  Output  Debug Console  Terminal  Ports                                   Python

PS C:\Users\meesa\OneDrive\Desktop\python activity> & C:\Users\meesa\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/meesa/OneDrive/Desktop/python activity/python4.py"
Opening stock: 100
Stock received: 50
Stock dispatched: 80
Recorded closing stock: 70
Expected stock: 70
Stock matches
PS C:\Users\meesa\OneDrive\Desktop\python activity> & C:\Users\meesa\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/meesa/OneDrive/Desktop/python activity/python4.py"
Opening stock: 60
Stock received: 20
Stock dispatched: 25
Recorded closing stock: 54
Expected stock: 55
Stock discrepancy: -1
```

CASE STUDY 05: Odd-Sized Symmetric Diamond Star Pattern Generator Question #05

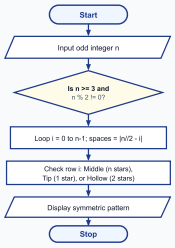
1. PROBLEM STATEMENT: Validate an odd integer greater than 2 and print a symmetric hollow diamond star pattern featuring a solid horizontal middle row.

2. OBJECTIVES: Read integer n • Validate n ≥ 3 and odd • Compute outer and inner spaces per row • Print solid middle row and hollow diamond edges.

3. ASSUMPTIONS: Input n must be an odd integer ≥ 3. Top and bottom rows have a single centered star, middle row (i == n // 2) has n stars, and other rows are hollow.

4. ALGORITHM: 1. Start → 2. Input n → 3. If n < 3 or n%2 == 0 print error → 4. Else loop i from 0 to n−1 → 5. spaces = abs(n//2 − i) → 6. Print solid center, single tip, or hollow row → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

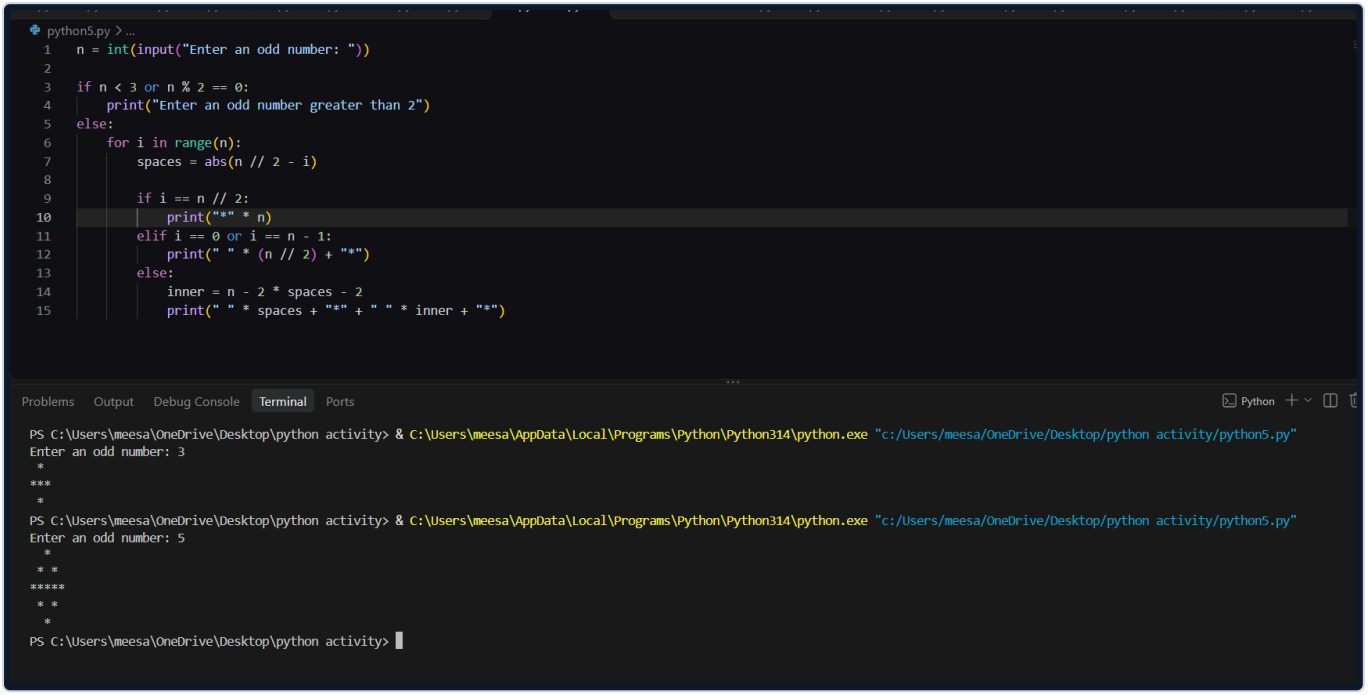
```
BEGIN INPUT n IF n < 3 OR n % 2 == 0 THEN PRINT "Enter an odd number > 2" ELSE FOR i = 0 TO n - 1 DO spaces = abs(n // 2 - i) IF i == n // 2 THEN PRINT "*" * n ELSE IF i==0 OR i==n-1 THEN PRINT tip "*" ELSE PRINT outer spaces + "*" + inner + "*" ENDFOR ENDIF END
```

7. PYTHON SOURCE CODE

```
n = int(input("Enter an odd number: ")) if n < 3 or n % 2 == 0: print("Enter an odd number greater than 2") else: for i in range(n): spaces = abs(n // 2 - i) if i == n // 2: print("*" * n) elif i == 0 or i == n - 1: print(" " * (n // 2) + "*") else: inner = n - 2 * spaces - 2 print(" " * spaces + "*" + " " * inner + "*")
```

8. TEST CASE: **Sample Input:** Run 1: 3 | Run 2: 5 | **Expected Output:** 3-row and 5-row hollow star diamonds with solid middle lines *** and *****.

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program validates that the input size is an odd number greater than 2 and prints a symmetric star diamond with a solid center bar.

CASE STUDY 06: Automated Quiz Evaluation & Negative Marking

Question #06

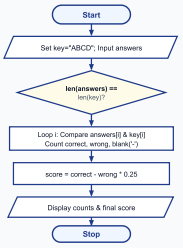
1. PROBLEM STATEMENT: Compare a student's quiz response string with an answer key, count correct, wrong, and unanswered items, and compute final score with negative marking.

2. OBJECTIVES: Read student answers and convert to uppercase • Validate answer length • Count correct, wrong, and blank ('-') responses • Apply −0.25 per wrong answer.

3. ASSUMPTIONS: Answer key is "ABCD". Each correct answer gives +1 mark, each wrong answer deducts 0.25 marks, and '-' indicates an unanswered question (0 marks).

4. ALGORITHM: 1. Start → 2. Set key="ABCD" & input answers → 3. If len(answers) != len(key) print error → 4. Loop i across key: count blank, correct, wrong → 5. score = correct − wrong*0.25 → 6. Display counts & score → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

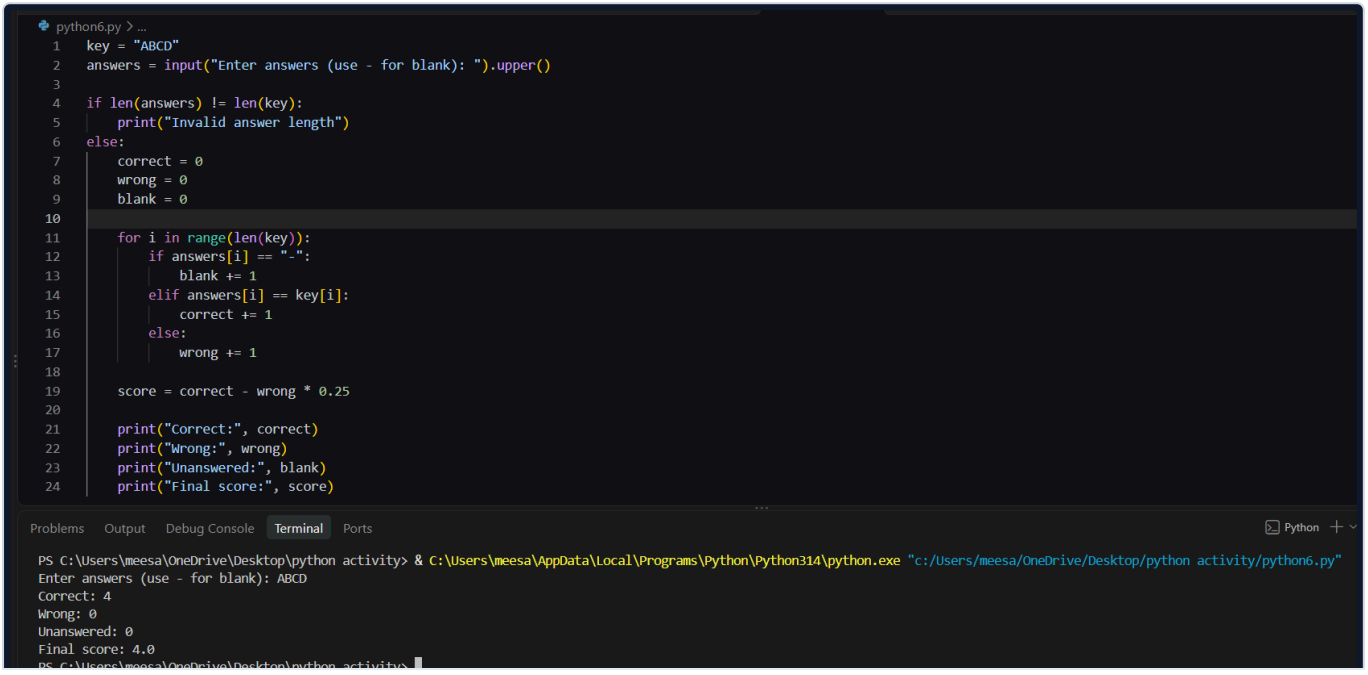
```
BEGIN key = "ABCD" INPUT answers IF len(answers) != len(key) THEN PRINT "Invalid answer length" ELSE correct = 0; wrong = 0; blank = 0 FOR i = 0 TO len(key) - 1 DO IF answers[i] == "-" THEN blank += 1 ELSE IF answers[i] == key[i] THEN correct += 1 ELSE wrong += 1 ENDFOR score = correct - wrong * 0.25 PRINT correct, wrong, blank, score ENDIF END
```

7. PYTHON SOURCE CODE

```
key = "ABCD" answers = input("Enter answers (use - for blank): ").upper() if len(answers) != len(key): print("Invalid answer length") else: correct = 0; wrong = 0; blank = 0 for i in range(len(key)): if answers[i] == "-": blank += 1 elif answers[i] == key[i]: correct += 1 else: wrong += 1 score = correct - wrong * 0.25 print("Correct:", correct) print("Wrong:", wrong) print("Unanswered:", blank) print("Final score:", score)
```

8. TEST CASE: **Sample Input:** Enter answers = ABCD | **Expected Output:** Correct: 4, Wrong: 0, Unanswered: 0, Final score: 4.0.

9. CODE AND OUTPUT SCREENSHOT



CASE STUDY 07: Monthly Calendar Grid Generator

Question #07

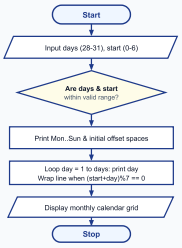
1. PROBLEM STATEMENT: Validate the number of days in a month (28–31) and starting weekday (0=Mon to 6=Sun), and print an aligned 7-column monthly calendar grid.

2. OBJECTIVES: Read days in month and starting weekday index • Validate ranges • Print weekday header and initial blank spaces • Wrap lines after every 7th column.

3. ASSUMPTIONS: Valid month days are 28 to 31 inclusive, and starting weekday is 0 (Monday) through 6 (Sunday).

4. ALGORITHM: 1. Start → 2. Input days & start → 3. If days not in 28..31 or start not in 0..6 print "Invalid input" → 4. Print header & initial offset spaces → 5. Loop day = 1 to days, printing newline when (start+day)%7 == 0 → 6. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN INPUT days, start IF days < 28 OR days > 31 OR start < 0 OR start > 6 THEN PRINT "Invalid input" ELSE PRINT "Mon Tue Wed Thu Fri Sat Sun" PRINT " " * start FOR day = 1 TO days DO PRINT day formatted IF (start + day) % 7 == 0 THEN PRINT newline ENDFOR ENDIF END
```

7. PYTHON SOURCE CODE

```
days = int(input("Enter number of days (28-31): ")) start = int(input("Starting weekday (0=Mon, 6=Sun): ")) if days < 28 or days > 31 or start < 0 or start > 6: print("Invalid input") else: print("Mon Tue Wed Thu Fri Sat Sun") print(" " * start, end="") for day in range(1, days + 1): print(f"{day:3}", end="") if (start + day) % 7 == 0: print() if (start + days) % 7 != 0: print()
```

8. TEST CASE: **Sample Input:** Days = 28, Starting weekday = 0 (Monday) | **Expected Output:** Aligned 4-week, 28-day calendar starting on Monday.

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program validates calendar constraints and prints a clean 7-column monthly grid with accurate weekday alignment.

CASE STUDY 08: Student Marks Analysis & Top Performer Finder

Question #08

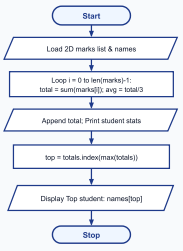
1. PROBLEM STATEMENT: Process a 2D list of student subject marks, calculate total and average marks for each student, and identify the top-scoring student.

2. OBJECTIVES: Load 2D marks list and student names • Compute total and average marks per student • Store totals in a list • Identify and print the top scorer.

3. ASSUMPTIONS: Sample dataset contains 3 subject scores each for Asha, Ravi, and Priya. All subject scores are valid numbers.

4. ALGORITHM: 1. Start → 2. Initialize marks, names, totals=[] → 3. Loop i over students: total = sum(marks[i]), avg = total/len(marks[i]) → 4. Append total & print student stats → 5. top = totals.index(max(totals)) → 6. Print names[top] → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN marks = [[80,75,90], [70,85,80], [95,90,88]] names = ["Asha", "Ravi", "Priya"] totals = [] FOR i = 0 TO len(marks) - 1 DO total = sum(marks[i]) average = total / len(marks[i]) APPEND total TO totals PRINT names[i], total, average ENDFOR top = index of max(totals) PRINT "Top student:", names[top] END
```

7. PYTHON SOURCE CODE

```
marks = [ [80, 75, 90], [70, 85, 80], [95, 90, 88] ] names = ["Asha", "Ravi", "Priya"] totals = [] for i in range(len(marks)): total = sum(marks[i]) average = total / len(marks[i]) totals.append(total) print(names[i], "Total:", total) print("Average:", average) top = totals.index(max(totals)) print("Top student:", names[top])
```

8. TEST CASE: **Sample Dataset:** Asha `[80,75,90]`, Ravi `[70,85,80]`, Priya `[95,90,88]` | **Expected Output:** Top student: `Priya` (Total: 273, Avg: `91.0`).

9. CODE AND OUTPUT SCREENSHOT



10. CONCLUSION: The program iterates over nested student score lists, computes individual totals and averages, and

CASE STUDY 09: Product Stock Valuation & Reorder Calculator

Question #09

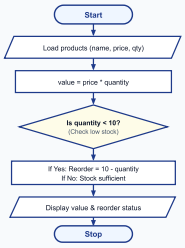
1. PROBLEM STATEMENT: Unpack product tuples containing name, unit price, and quantity, calculate stock valuation, and determine reorder quantities for items below 10 units.

2. OBJECTIVES: Load list of product tuples • Calculate stock value = price × quantity • Check if quantity < 10 • Print reorder quantity (10 − quantity) or sufficient status.

3. ASSUMPTIONS: Minimum required stock level per item is 10 units. Sample catalog consists of Pen, Book, and Bag tuples.

4. ALGORITHM: 1. Start → 2. Load products list → 3. For each (name, price, quantity): compute value = price * quantity → 4. Print name and value → 5. If quantity < 10 print reorder (10 − quantity), else print "Stock sufficient" → 6. Stop.

5. FLOWCHART



6. PSEUDOCODE

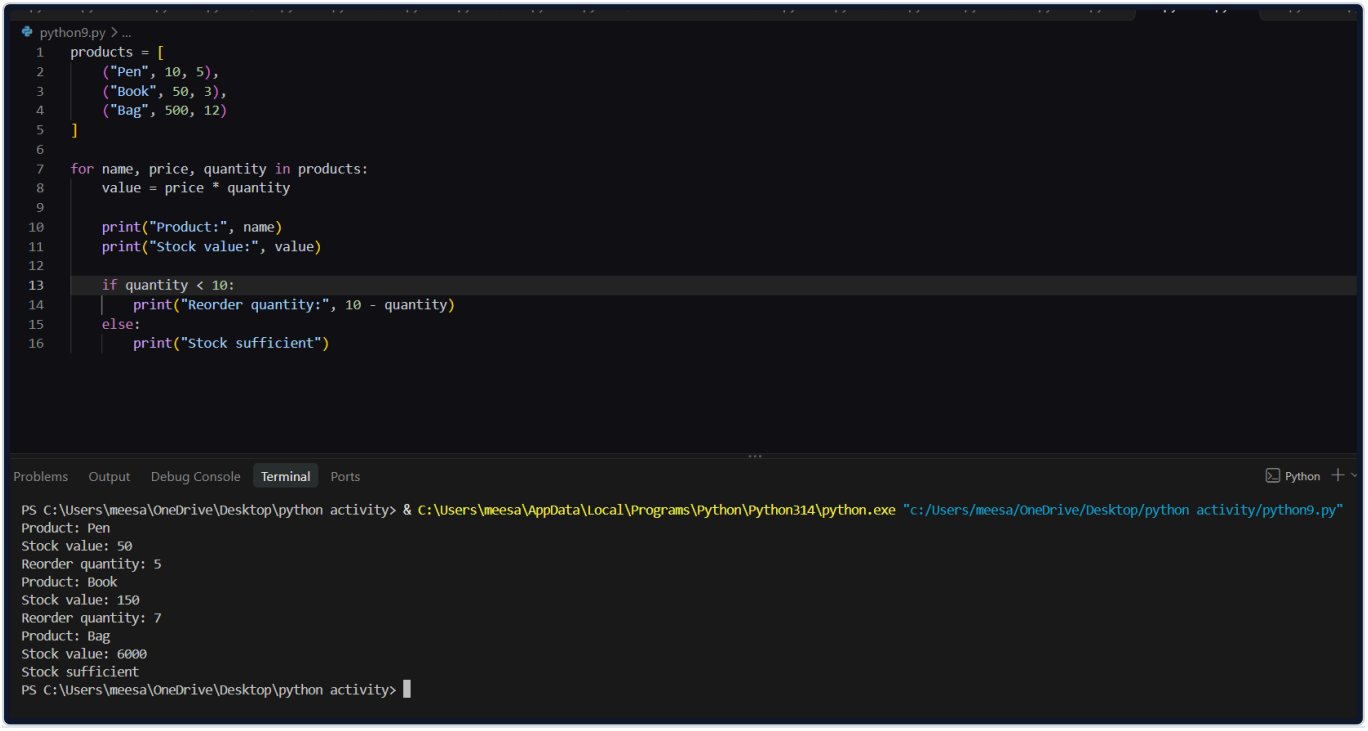
```
BEGIN products = [("Pen",10,5), ("Book",50,3), ("Bag",500,12)] FOR EACH (name, price, quantity) IN products DO value = price * quantity PRINT "Product:", name PRINT "Stock value:", value IF quantity < 10 THEN PRINT "Reorder quantity:", 10 - quantity ELSE PRINT "Stock sufficient" ENDIF ENDFOR END
```

7. PYTHON SOURCE CODE

```
products = [ ("Pen", 10, 5), ("Book", 50, 3), ("Bag", 500, 12) ] for name, price, quantity in products: value = price * quantity print("Product:", name) print("Stock value:", value) if quantity < 10: print("Reorder quantity:", 10 - quantity) else: print("Stock sufficient")
```

8. TEST CASE: **Sample Dataset:** `("Pen",10,5), ("Book",50,3), ("Bag",500,12)` | **Expected Output:** Pen (Val 50, Reorder 5), Book (Val 150, Reorder 7), Bag (Val 6000, Sufficient).

9. CODE AND OUTPUT SCREENSHOT



CASE STUDY 10: Cafeteria Food Waste & Leftover Percentage Analyzer Question #10

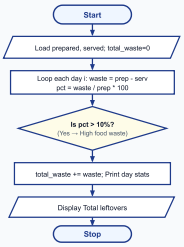
1. PROBLEM STATEMENT: Analyze daily prepared and served food quantities, calculate daily leftovers and waste percentages, flag days above 10% waste, and sum total leftovers.

2. OBJECTIVES: Load daily prepared and served lists • Compute daily waste and percentage • Flag days where waste > 10% • Accumulate and display total leftovers.

3. ASSUMPTIONS: Prepared quantities are positive and ≥ served quantities. Daily waste percentage > 10% outputs a "High food waste" alert.

4. ALGORITHM: 1. Start → 2. Load prepared & served lists, total_waste = 0 → 3. Loop i over days: waste = prepared[i] − served[i], pct = waste/prepared[i]*100 → 4. Add waste to total_waste & print stats → 5. If pct > 10 print alert → 6. Print total_waste → 7. Stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN prepared = [100, 120, 90]; served = [90, 100, 85] total_waste = 0 FOR i = 0 TO len(prepared) - 1 DO waste = prepared[i] - served[i] pct = (waste / prepared[i]) * 100 total_waste = total_waste + waste PRINT Day i+1, waste, round(pct, 2) IF pct > 10 THEN PRINT "High food waste" ENDFOR PRINT "Total leftovers:", total_waste END
```

7. PYTHON SOURCE CODE

```
prepared = [100, 120, 90] served = [90, 100, 85] total_waste = 0 for i in range(len(prepared)): waste = prepared[i] - served[i] percentage = waste / prepared[i] * 100 total_waste += waste print("Day", i + 1) print("Leftovers:", waste) print("Waste percentage:", round(percentage, 2), "%") if percentage > 10: print("High food waste") print("Total leftovers:", total_waste)
```

8. TEST CASE: **Sample Dataset:** Prepared=[100,120,90], Served=[90,100,85] | **Expected Output:** Day 2 flags High food waste (16.67%); Total leftovers: 35.

9. CODE AND OUTPUT SCREENSHOT

